



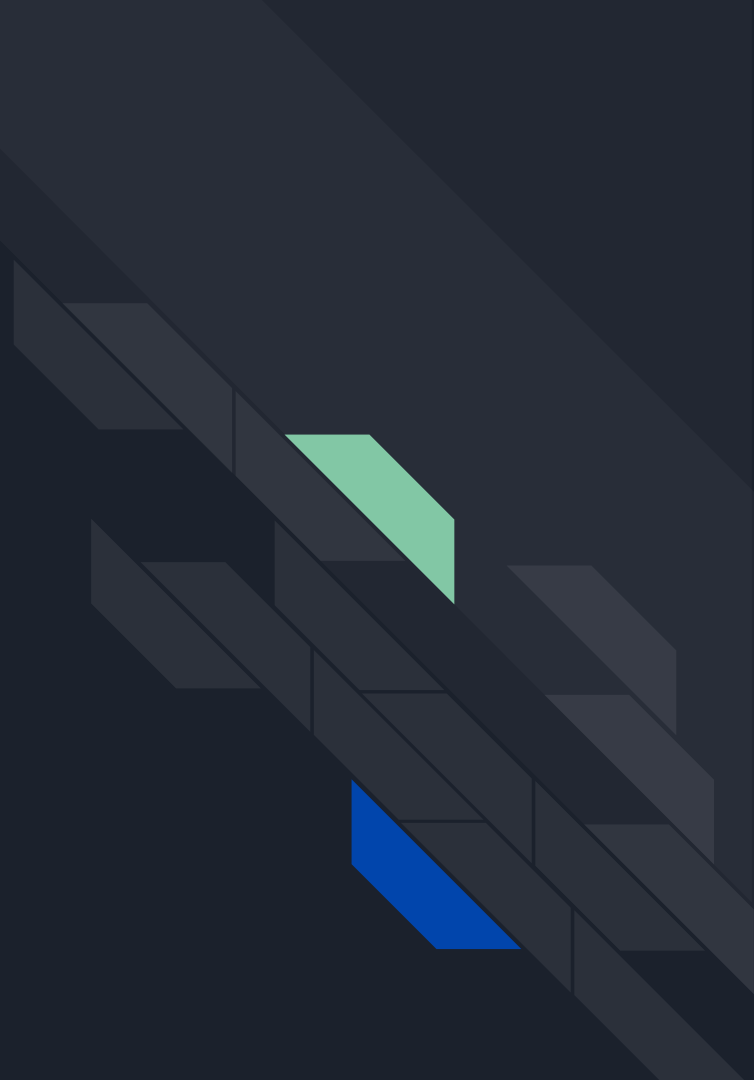
Zephyr + BYOB: Bring your own bootloader

Benedek Kupper

Embedded Software Engineer

Agenda

1. What Zephyr ships -
and what's missing
2. The necessary Zephyr toolkit:
modules and sysbuild
3. Bootloader Zephyr build integration
4. Bootloader - Zephyr application link
5. The rest of the system (non-Zephyr)





The role of a bootloader

The bootloader decides what image to boot, essential for a software upgradeable system.

- First piece of firmware to execute after a CPU reset
- Basic hardware initialization (e.g. external flash)
- Read boot context (e.g. update request, boot success flag) and image signatures
- Perform updates either through a transport or from secondary memory
- Validate application integrity and authenticity (checksum or hash)
- Jump to application



Bootloader operating modes

Launch update mode on request,
transfer new image, flash then boot

- + Allows single image slot layout
- + Update possible without app
- Failed update makes the system unusable

Choose the right image to boot from
multiple slots (and swap slots)

- + Fallback image always available
- + No need for transport stack
- Flash has to fit app twice



MCUboot - Zephyr's built-in bootloader

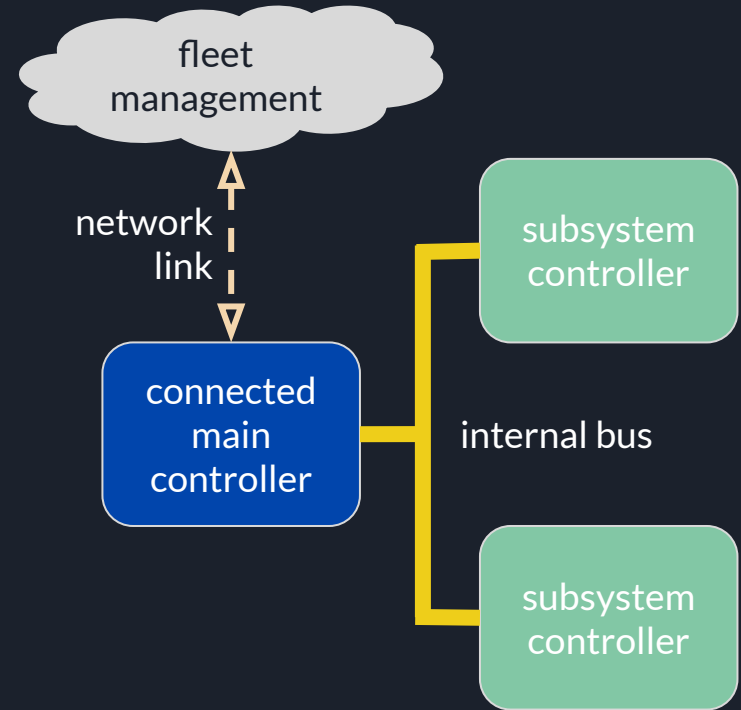
Open-source, cross-RTOS secure bootloader, adopted by Zephyr

- Secure boot through image signature validation
- Dual slot image management by swap / overwrite / X In Place / X in RAM
- Semantic versioning and anti-downgrade protection
- Committing after a test run: a new image must confirm valid operation before approved for booting another time
- Serial recovery mode: receive updates through UART / USB

(MCUmgr running in the Zephyr application receives and saves updates)

Case study - a distributed system

- Main controller performs secure OTA update, and updates subsystems
- Subsystem controller:
STM32 MCU with 64 kB flash
- Battery power ensures uninterrupted power during updates





Subsystem firmware update

Requirements

- Update over CAN bus
- Minimal bootloader code space (8 kB top)
- No need for fallback image, or cryptographic validation

MCUboot limitations

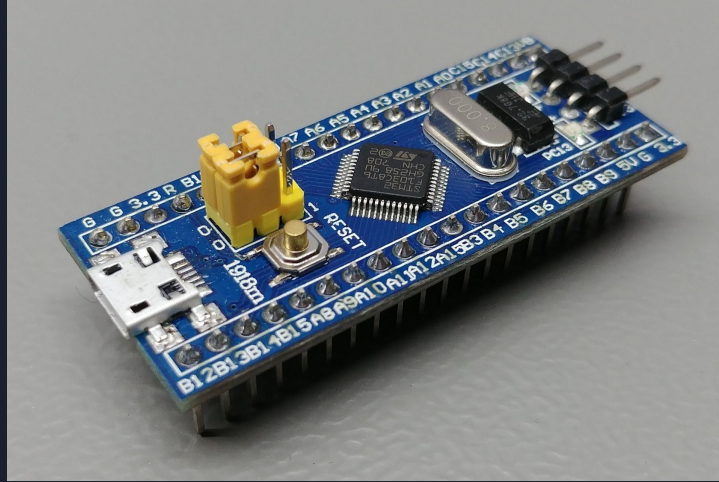
- Update only over UART / USB
- Large memory footprint (typically 32-64 kB flash)
- Double application image size for slot swapping

→ Design a custom bootloader



Bootloader design goals - KISS

- Minimal flash footprint -> simple, bare-metal implementation
- Shared, portable CAN firmware transport protocol
- CMake build and Kconfig integration for parameter configuration
- Easy system integration using sysbuild
- Catchy name :)



Blue Pill CAN bootloader



Sysbuild in Zephyr

Sysbuild is a CMake project within Zephyr ([share/sysbuild](#)):

- Tells CMake which actual images to build (app, MCUboot, etc)
- Uses its own Kconfig system to make decisions, including propagating settings to image builds, keeping them in sync
- Unlike Zephyr's Kconfig, Sysbuild's isn't device-tree aware
- Tracks image cross-dependencies and enforces correct flashing order
- Invoked with [west build --sysbuild](#) or using cmake at [share/sysbuild](#)



Zephyr modules

Modules are repositories that contain a `zephyr/module.yml` file and thus integrated into the build system automatically.

- Commonly pulled in either as west projects, or by cmake (using ZEPHYR_MODULES)
- Modules extend Zephyr's CMake, Kconfig and devicetree systems, including sysbuild
- Recommended reference:
<https://github.com/zephyrproject-rtos/example-application>



Sysbuild control knobs

Project scope:

- sysbuild.cmake
(~ CMakeLists.txt)
- Kconfig.sysbuild
(~ Kconfig)
- sysbuild.conf
(~ prj.conf)

Module scope (reusable):

referenced in zephyr/module.yml

```
build:  
  cmake: bluepill-can-bl  
  kconfig: zephyr/Kconfig  
  sysbuild-cmake: zephyr/sysbuild  
  sysbuild-kconfig: zephyr/sysbuild/Kconfig
```



Bootloader module structure

- CMakeLists.txt
 - bare-metal bootloader build target
- CMakePresets.json
 - standalone build configuration values
- zephyr
 - module.yml
 - Zephyr module definition
 - Kconfig
 - application Kconfig definitions
 - bluepill_partitions.overlay
 - flash layout for bootloader + application
 - sysbuild
 - CMakeLists.txt
 - Sysbuild bootloader build instructions
 - Kconfig
 - Sysbuild Kconfig definitions
- bluepill-can-bl
 - Kconfig
 - shared bootloader Kconfig definitions
 - CMakeLists.txt
 - defines application sources for interacting with the bootloader



Sysbuild flow

1. Sysbuild parses its own Kconfig settings, and finds `SB_CONFIG_BOOTLOADER_BLUEPILL_CAN=y`
2. Sysbuild runs CMake and adds the bootloader using `ExternalProject_Add()`, forwarding its relevant Kconfig options (and DT overlay) to bootloader and application
3. CMake builds the bootloader
4. CMake builds the Zephyr application
5. The two images are also merged into a single hex file using `HEX_FILES_TO_MERGE`



Application - bootloader interaction

- Single request from application to bootloader: try to perform an update
- This flag can be stored in flash / RAM / backup RAM
 - the latter is chosen: single source file built in both images – instead of having to share linker script fragments for a dedicated RAM location



Main (updater) controller side

The updater needs two artifacts from the Zephyr build:

- `include/generated/zephyr/app_version.h`
- `zephyr.bin`

It also uses the bootloader's Kconfig and CMake library definition for CAN update protocol

The update flow:

1. Fetch and compare app versions
2. Request boot to update mode
3. Transfer each flash page data as the bootloader requests
4. The update completes with the last shorter transfer



CAN bootloader in less than 2 kB

- C++20, register level programming instead of STM32 HAL
 - Even using arcane CPU-specific bit-addressing method (bit-band)
- CAN bit timing and filter register values are calculated at compile-time (C++ constexpr)
- Stripped down interrupt vector table for faults only
- Simple, custom update protocol



Questions?

PS: github.com/benedekkupper/bluepill-can-bl is FOSS